

gRPC

High Performance Communication for Modern Distributed Systems

From Google's internal infrastructure to the open-source standard powering Netflix, Uber, and Square — gRPC redefines how distributed systems communicate. Built on HTTP/2 and Protocol Buffers, it delivers speed, efficiency, and strong contracts at every scale.

BACKEND ENGINEERING

MICROSERVICES



Why Do We Need gRPC?

REST has served us well — but modern distributed systems expose its cracks. High latency, bloated payloads, and weak contracts become critical at scale.

The Problem with REST

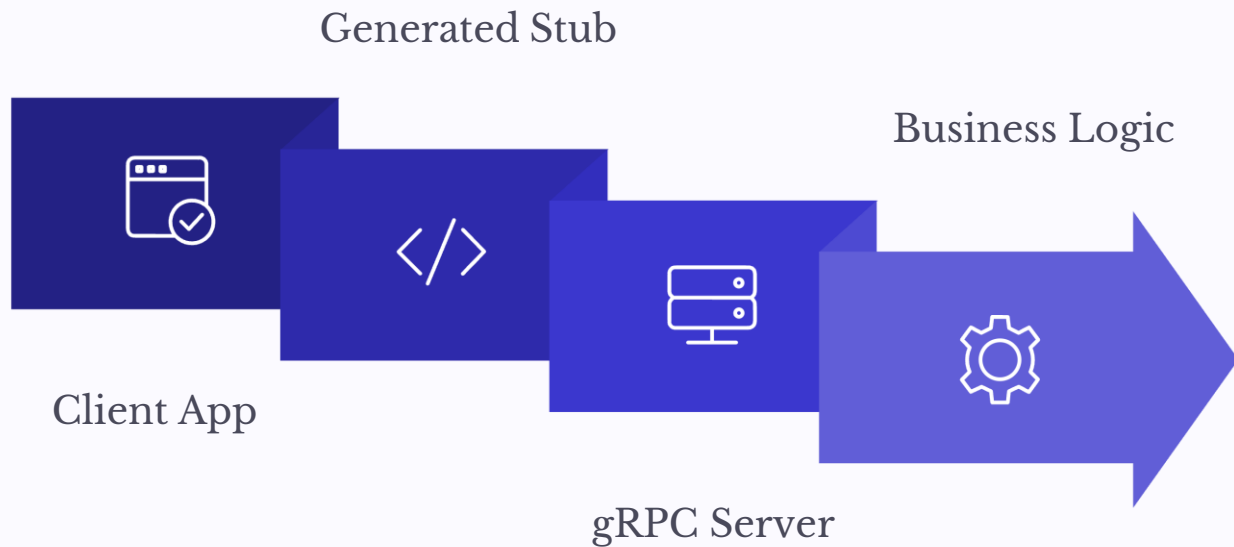
- **JSON overhead** — verbose text format inflates payloads
- **Slow serialization** — parsing text is slower than binary
- **Large payloads** — field names repeated in every response
- **Multiple HTTP connections** — no multiplexing on HTTP/1.1
- **Mobile latency** — chatty APIs drain battery and bandwidth
- **No streaming** — real-time data requires workarounds

The gRPC Solution

- **Binary Protocol Buffers** — compact, fast, schema-driven
- **HTTP/2 multiplexing** — one connection, many streams
- **Header compression** — HPACK reduces overhead
- **Built-in streaming** — unary, server, client, bidirectional
- **Strong API contracts** — .proto files as source of truth
- **Auto-generated code** — in 12+ languages

What Is gRPC?

gRPC is a modern, open-source **Remote Procedure Call (RPC)** framework originally developed at Google. It enables services to call each other as if they were local functions — across networks, languages, and infrastructure boundaries.



REST vs gRPC

Both solve service communication — but they optimize for very different worlds. REST excels at public, human-readable APIs. gRPC dominates internal, high-performance microservice communication.

Dimension	REST / JSON	gRPC / Protobuf
Transport	HTTP/1.1 (mostly)	HTTP/2 (required)
Data Format	JSON (text)	Protocol Buffers (binary)
Payload Size	Large — field names repeated	Compact — binary encoding
Speed	Slower serialization	3–10× faster
Streaming	Limited (SSE, WebSockets)	Native — all 4 types
Browser Support	Universal	Limited (needs grpc-web)
API Contract	Informal (OpenAPI optional)	Strict (.proto required)
Code Generation	Manual or via OpenAPI	Auto-generated, type-safe
Learning Curve	Low — widely known	Moderate — proto + HTTP/2
Best For	Public APIs, web browsers	Internal microservices

 **Rule of thumb:** Use REST for public-facing APIs and browser clients. Use gRPC for internal service-to-service communication where performance and contracts matter.

HTTP/2 — The Transport Layer

gRPC is built exclusively on **HTTP/2** — a binary, multiplexed protocol that solves the fundamental limitations of HTTP/1.1. A single TCP connection carries all requests and responses simultaneously.

→ Binary Protocol

Frames are binary, not text — faster to parse, less error-prone

→ Multiplexing

Multiple streams share one connection — no head-of-line blocking

→ Header Compression

HPACK eliminates redundant header overhead on every request

→ Server Push

Server can proactively send data the client hasn't requested yet

HTTP/1.1 vs HTTP/2

HTTP/1.1: One request per connection at a time. Multiple connections needed for parallelism. Text-based headers repeated on every request.

HTTP/2: One persistent connection. Many simultaneous streams. Binary frames. Compressed headers. Lower latency by design.

gRPC leverages all of these — making it ideal for high-throughput microservice meshes.

Protocol Buffers — The Data Layer

Protocol Buffers (Protobuf) are Google's language-neutral, platform-neutral mechanism for serializing structured data. Think of them as JSON — but smaller, faster, and strongly typed.

Why Protobuf?

Compact Binary

3–10× smaller than equivalent JSON — no field name repetition

Schema-Driven

.proto files define structure, types, and versioning rules

Forward/Backward Compatible

Add fields without breaking existing clients using field numbers

Auto-Generated Code

Compile .proto → native classes in Java, Go, Python, and more

JSON vs Protobuf — Size Comparison

```
{  
  "user_id": 12345,  
  "name": "Alice Chen",  
  "email": "alice@example.com",  
  "active": true,  
  "score": 98.5  
}
```

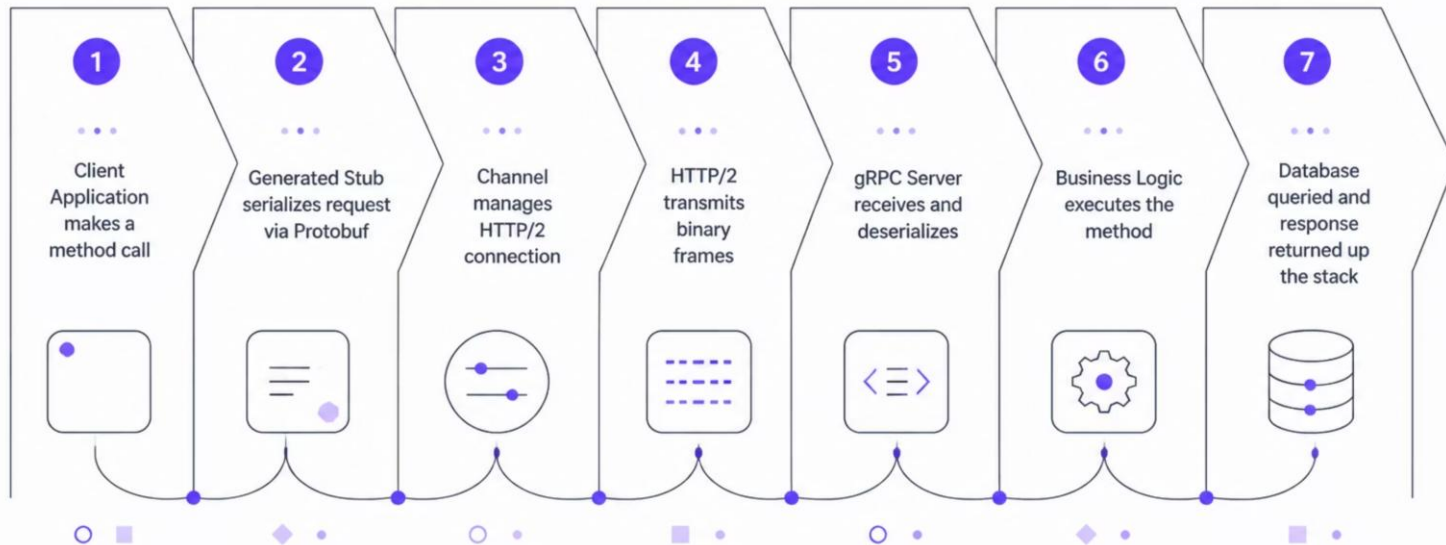
JSON: **~120 bytes** (text, field names repeated)

```
// Protobuf binary encoding  
// Same data: ~35 bytes  
// Field numbers, not names  
// Varint encoding for integers
```

Protobuf: **~35 bytes — 70% smaller**

gRPC Architecture — End to End

Every gRPC call follows a clean, layered architecture. The client and server never directly talk to each other — generated code and HTTP/2 handle all the plumbing.



The flow is symmetrical: requests travel down the client stack, across the wire, and up the server stack. Responses follow the reverse path. Deadlines, metadata, and interceptors can be injected at the Channel or Stub layer without touching business logic.

Key Terminology

Master these terms and you'll speak gRPC fluently in any technical interview or architecture review.



.proto File

The source-of-truth schema defining services, messages, and RPC methods



Service

A collection of RPC methods — analogous to a REST controller or interface



Stub

Auto-generated client proxy — you call methods on it, it handles serialization



Channel

Manages the underlying HTTP/2 connection, load balancing, and retries



Message

A typed data structure defined in .proto — serialized as binary Protobuf



Stream

A persistent channel for sending multiple messages — supports all 4 RPC types



Deadline

Maximum time allowed for an RPC to complete — prevents hanging calls



Metadata

Key-value pairs attached to calls — used for auth tokens, tracing, routing



Status Codes

Standardized error codes (OK, NOT_FOUND, DEADLINE_EXCEEDED, etc.)

Anatomy of a .proto File

Complete Example

```
syntax = "proto3";           // proto3 is standard

package com.example.user; // namespace

option java_package = "com.example.user.proto";
option java_multiple_files = true;

// Message = data structure
message UserRequest {
    int32 user_id = 1;        // field number = binary key
    string locale = 2;
}

message UserResponse {
    string name = 1;
    string email = 2;
    bool active = 3;
}

// Service = API interface
service UserService {
    // Unary RPC
    rpc GetUser(UserRequest)
        returns (UserResponse);

    // Server streaming
    rpc ListUsers(UserRequest)
        returns (stream UserResponse);

    // Client streaming
    rpc CreateUser(stream UserRequest)
        returns (UserResponse);

    // Bidirectional streaming
    rpc SyncUsers(stream UserRequest)
        returns (stream UserResponse);
}
```

Line-by-Line Breakdown

01

syntax = "proto3"

Declares the version. proto3 is the current standard — simpler than proto2

02

package + java_package

Namespace for generated code — prevents class name collisions

03

message blocks

Define data structures. Each field has a type, name, and unique number

04

service + rpc

Define the API contract. rpc declares method name, input, and output types

05

stream keyword

Prefix on input or output to enable streaming — independently on either side



Field numbers are permanent. Never reuse or reorder them — they are the binary wire identifier, not the field name.

Types of RPC & Java Implementation

gRPC supports four RPC patterns — from simple request/response to full-duplex streaming. Each maps to a distinct method signature in generated code.



Unary RPC

One request → one response. Classic RPC.
Example: GetUser(id)



Server Streaming

One request → stream of responses. *Example: ListUsers()* → stream



Client Streaming

Stream of requests → one response. *Example: UploadFile(chunk)*



Bidirectional Streaming

Both sides stream independently. *Example: Chat, live sync*

Server Implementation (Java)

```
public class UserServiceImpl
    extends UserServiceGrpc.UserServiceImplBase {

    @Override
    public void getUser(
        UserRequest req,
        StreamObserver<UserResponse> obs) {

        User user = repo.findById(req.getUserId());
        UserResponse res = UserResponse.newBuilder()
            .setName(user.getName())
            .setEmail(user.getEmail())
            .build();
        obs.onNext(res);
        obs.onCompleted();
    }
}
```

Client Call (Java)

```
ManagedChannel channel =
    ManagedChannelBuilder
        .forAddress("localhost", 9090)
        .usePlaintext()
        .build();

UserServiceGrpc.UserServiceBlockingStub stub =
    UserServiceGrpc.newBlockingStub(channel);

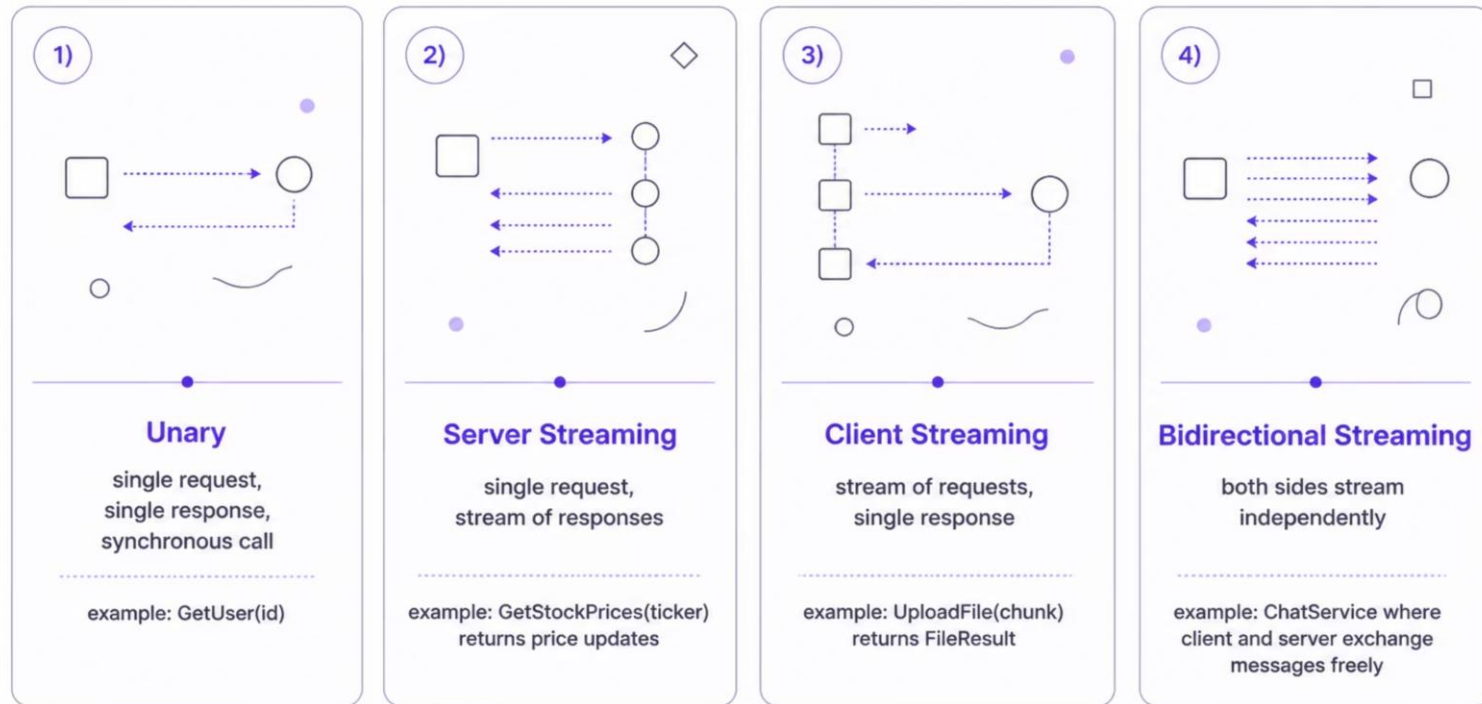
UserRequest req = UserRequest.newBuilder()
    .setUserId(12345)
    .build();

UserResponse res = stub.getUser(req);
System.out.println(res.getName());

channel.shutdown();
```

The Four RPC Patterns

gRPC supports four communication patterns — each suited to different workloads. Understanding when to use which pattern is key to designing efficient APIs.



Unary RPC — Simple Request / Response

The most common pattern. The client sends a single request and waits for a single response — like a traditional HTTP REST call, but over HTTP/2 with Protocol Buffers.

.proto Definition

```
service UserService {  
  rpc GetUser (UserRequest)  
    returns (UserResponse);  
}
```

Use cases: Fetching a user profile, validating a token, submitting a form, retrieving a product catalog entry.

Sequence Flow



Simple, synchronous, and blocking by default. Ideal for CRUD-style operations where a single result is expected.

Server Streaming RPC — One Request, Many Responses

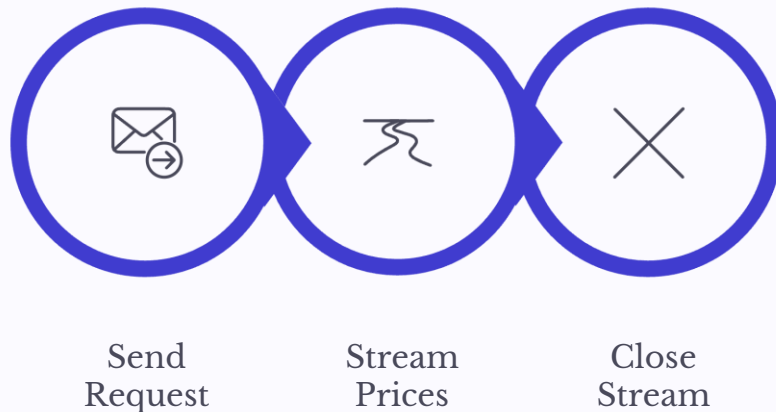
The client sends a single request, and the server responds with a **stream of messages**. The connection stays open until the server finishes or the client cancels.

.proto Definition

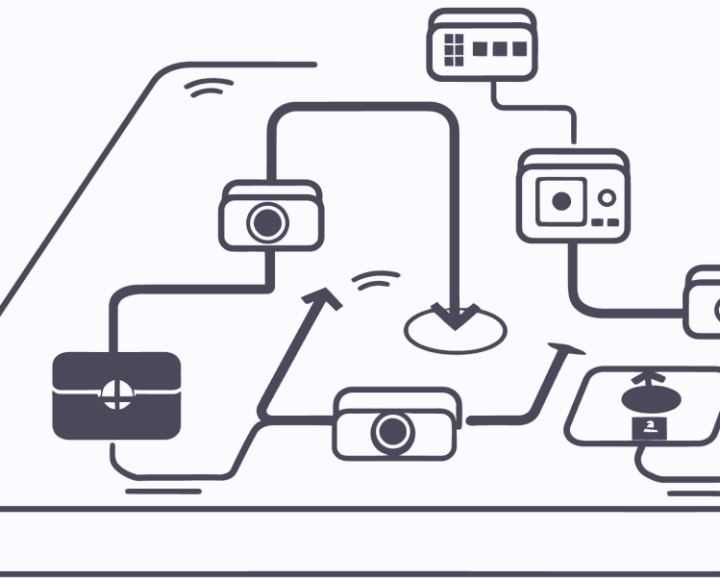
```
service StockService {  
  rpc GetPrices (StockRequest)  
    returns (stream PriceUpdate);  
}
```

Use cases: Real-time stock prices, log tailing, sensor telemetry, progress notifications, live sports scores.

Sequence Flow



The server controls the stream lifecycle. The client reads messages as they arrive without re-establishing connections.



CLIENT STREAMING

Client Streaming RPC — Many Requests, One Response

The client sends a **stream of messages** to the server, which processes them and returns a single aggregated response once the stream closes.

.proto Definition

```
service UploadService {  
  rpc UploadFile (stream FileChunk)  
    returns (UploadResult);  
}
```

Real-World Use Cases

- Uploading large files in chunks
- Batch processing job submissions
- Sending sensor data aggregates
- Voice or video frame ingestion

Flow

Client opens stream → sends chunks → server acknowledges with single `UploadResult` containing status, checksum, and storage path.

Bidirectional Streaming — Full-Duplex Communication

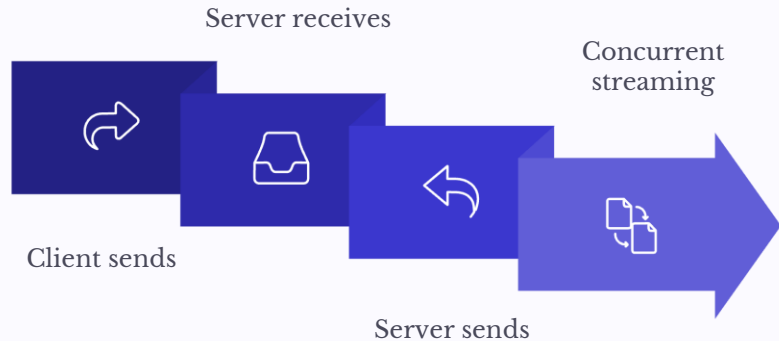
Both client and server open independent streams and send messages **simultaneously**. Neither side waits — ideal for interactive, real-time systems.

.proto Definition

```
service ChatService {  
  rpc Chat (stream ChatMessage)  
    returns (stream ChatMessage);  
}
```

Use cases: Chat applications, multiplayer gaming, collaborative editing, live trading platforms, IoT command/control.

Sequence Flow



Each side reads and writes independently. Streams can be closed by either party without affecting the other's ability to send.

Java Implementation — Spring Boot gRPC

gRPC with Spring Boot uses the `grpc-spring-boot-starter` library. Service implementations extend generated abstract classes, and clients use managed channels with interceptors.

Server — Service Implementation

```
@GrpcService
public class UserServiceImpl
    extends UserServiceGrpc.UserServiceImplBase {

    @Override
    public void getUser(
        UserRequest req,
        StreamObserver<UserResponse> obs) {

        User user = repo.findById(req.getId());
        UserResponse resp = UserResponse.newBuilder()
            .setId(user.getId())
            .setName(user.getName())
            .build();
        obs.onNext(resp);
        obs.onCompleted();
    }
}
```

Client — Channel + Stub

```
@Bean
public ManagedChannel channel() {
    return ManagedChannelBuilder
        .forAddress("localhost", 9090)
        .usePlaintext()
        .intercept(new AuthInterceptor())
        .build();
}

@Bean
public UserServiceGrpc.UserServiceBlockingStub
    userStub(ManagedChannel channel) {
    return UserServiceGrpc
        .newBlockingStub(channel);
}

// Usage
UserResponse resp = userStub
    .getUser(UserRequest.newBuilder()
        .setId(123).build());
```

❑ Use `newBlockingStub()` for sync calls, `newFutureStub()` for async, or `newStub()` for full streaming.

Protocol Buffers — The .proto Contract

The `.proto` file is the single source of truth. It defines messages, services, and RPC methods. The `protoc` compiler generates Java classes automatically — no manual serialization code.

Complete .proto Example

```
syntax = "proto3";
package com.example.user;

message UserRequest {
  int32 id = 1;
}

message UserResponse {
  int32 id = 1;
  string name = 2;
  string email = 3;
}

service UserService {
  // Unary
  rpc GetUser (UserRequest)
    returns (UserResponse);

  // Server Streaming
  rpc ListUsers (UserRequest)
    returns (stream UserResponse);

  // Client Streaming
  rpc CreateUser (stream UserResponse)
    returns (UserRequest);

  // Bidirectional
  rpc Chat (stream UserResponse)
    returns (stream UserResponse);
}
```

Generated Java Classes

```
// Auto-generated by protoc
UserRequest req = UserRequest.newBuilder()
    .setId(42)
    .build();

int id = req.getId();           // getter
UserRequest copy = req
    .toBuilder()
    .setName("Alice")
    .build();

// Builder pattern, immutable, thread-safe
```

- ❑ Classes are **immutable**, **thread-safe**, and use the **builder pattern**. Field numbers (= 1, = 2) are critical for binary compatibility — never reorder them.

Advantages & Limitations

gRPC excels in backend-to-backend microservices but has trade-offs. Understanding both sides helps you choose the right tool for each job.

✓ Advantages

- **High Performance** — Binary serialization is 3–10× faster than JSON
- **HTTP/2** — Multiplexing, header compression, low latency
- **Streaming** — All four RPC patterns natively supported
- **Strong Contracts** — Schema-first with backward-compatible evolution
- **Multi-Language** — Java, Go, Python, C++, Node, Rust, and more
- **Auto Code Generation** — No boilerplate serialization logic
- **Cloud Native** — Native Kubernetes, Istio, and Envoy integration

⚠ Limitations

- **Browser Support** — Requires grpc-web proxy; no native browser support
- **Debugging** — Binary payloads aren't human-readable like JSON
- **Learning Curve** — Proto schemas, code generation, and streaming semantics
- **Proxy / Firewall** — Some infrastructure blocks HTTP/2 or long-lived streams
- **Versioning** — Schema evolution requires careful field management

❏ **Use REST when:** public APIs, browser clients, simple CRUD, or human-readable payloads are required.